Group 1 – Multithreading & Multiprocessing:

Explain the difference between multithreading and multiprocessing in Python.

Create a program that demonstrates both concepts and compare their performance with explanation.

Multithreading vs Multiprocessing in Python

Part 1: Introduction & Key Concepts

Multithreading and multiprocessing are two fundamental approaches to concurrent programming in Python. Both allow multiple tasks to execute, but they differ significantly in their implementation and performance characteristics.

Definition

Multithreading: Multiple threads within a single process share the same memory space. Threads are lightweight and are managed by the operating system.

Multiprocessing: Multiple processes run independently with separate memory spaces. Each process has its own Python interpreter and can truly execute in parallel on multi-core systems.

Key Difference: The GIL (Global Interpreter Lock)

Python's GIL prevents true parallel execution of threads in CPU-bound tasks. This means multithreading threads cannot run Python code simultaneously, even on multi-core processors. Multiprocessing bypasses this limitation by using separate processes, each with its own GIL.

Part 2: Performance Comparison Analysis

Test Setup:

An example when we use python with both multithreading and multiprocessing

Task Type	Normal (seq)	Multithreading	Multiprocessing
Small Task (10K)	0.001s	0.004s	0.135s
Medium Task (100K)	0.012s	0.013s	0.008s
Large Task (1M)	0.120s	0.120s	0.035s

Comparison Metrics:

Metric	Multithreading	Multiprocessing
Memory Usage	Low (shared)	High (separate)
Startup Time	~0.001s	~0.13s
CPU Task Performance	Limited (GIL)	Excellent
I/O Task Performance	Excellent	Good
Inter-process Communication	Easy	Complex

Part 3: Practical Recommendations & Best Practices

When to Use Multithreading

✔ Task is I/O-bound

I/O-bound tasks spend most of their time **waiting** for input/output operations to complete rather than using the CPU.

✔ Low memory usage needed

All threads share the **same memory space**

No need to duplicate data

Threads are **lightweight** compared to processes

✔ Fast response required

Multithreading improves **responsiveness**, especially in applications where users should not feel delays.

When to Use Multiprocessing

✔ Task is CPU-bound:

CPU-bound tasks spend most of their time **performing calculations**, not waiting.

Examples:

- Mathematical computations
- Data processing
- Sorting large datasets
- Machine learning algorithms

✔ Heavy computation:

Each process has its **own Python interpreter**
Not restricted by GIL
Can fully utilize modern multi-core CPUs

✔ Multi-core utilization required:

Modern CPUs have multiple cores, but:

- Threads cannot use all cores due to GIL
- Processes **can**

Conclusion

The choice between multithreading and multiprocessing depends on your specific use case:

- Multithreading → Best for **I/O-bound tasks**
- Multiprocessing → Best for **CPU-bound tasks**
- Python GIL limits multithreading performance
- Multiprocessing provides **true parallelism**